

Time Complexity

Time complexity is a way to measure how the running time of an algorithm grows as the size of the input increases.

Simple Example

Suppose we want to check if a number exists in a list.

Linear Search

```
bool findNumber(int arr[], int n, int x) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == x)  
            return true;  
    }  
    return false;  
}
```

Example 1: Linear Search

Suppose you have an array of size n .

To find an element, you may have to check every element one by one.

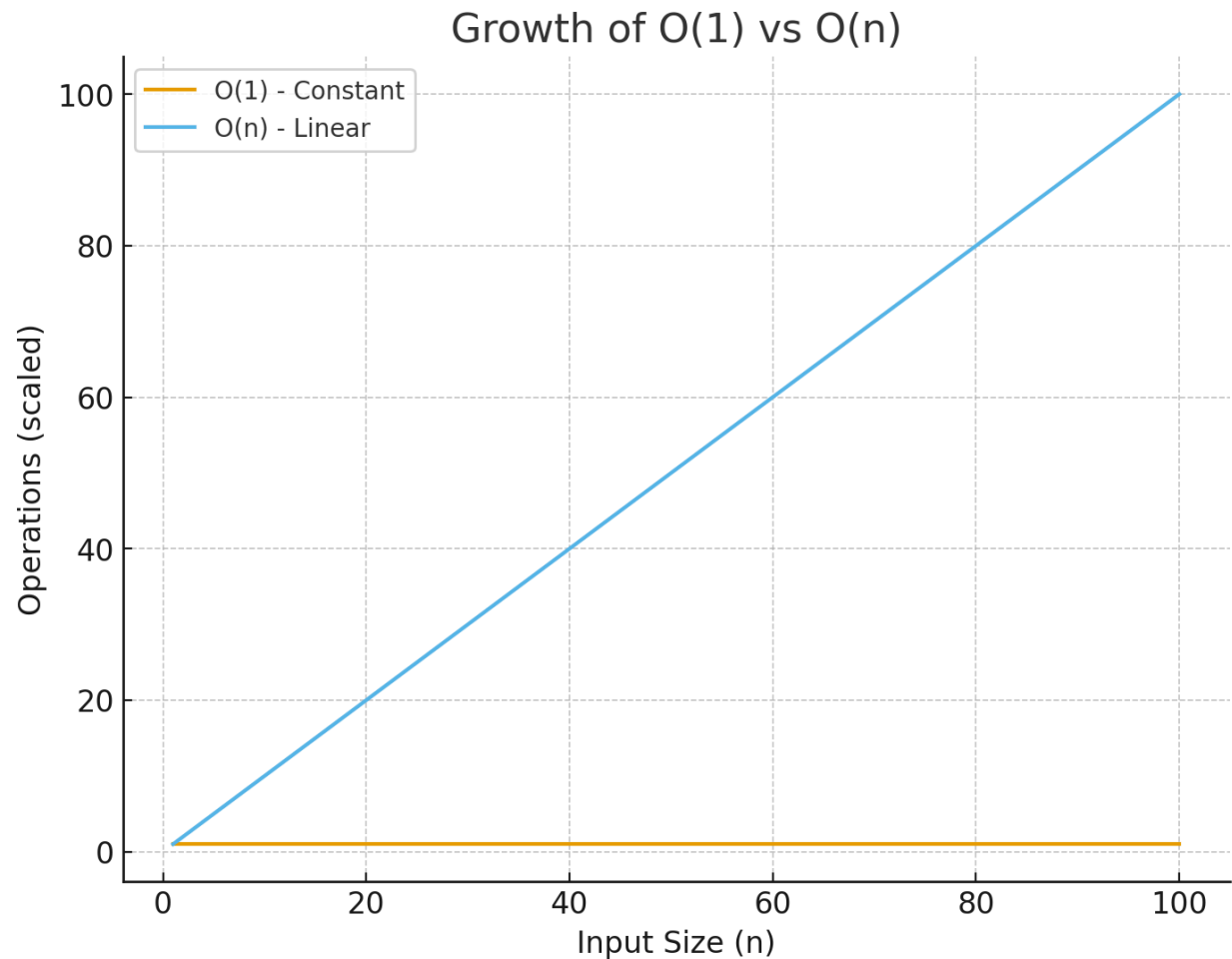
If $n = 10$, max 10 checks.

If $n = 1,000$, max 1,000 checks.

If $n = 1,000,000$, max 1,000,000 checks.

So the time grows **directly with $n \rightarrow O(n)$** .

In short, **Time complexity** tells us the pattern of growth, not the exact time.



Here's the graph comparing only $O(1)$ and $O(n)$:

- $O(1)$: stays constant no matter how big the input gets.
- $O(n)$: increases linearly with input size.

Commonly Used Time Complexities

1. $O(1)$ – Constant Time

Formula-based calculation takes the same time regardless of input size.

Example: Sum of first n natural numbers

```
int sumOfNaturalNumbers(int n) {  
    return (n * (n + 1)) / 2; // Single formula, constant time  
}
```

2. O(n) – Linear Time

Time grows directly with input size.

Example: Print elements of an array

```
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " "; // Runs n times  
    }  
}
```

3. O(n²) – Quadratic Time

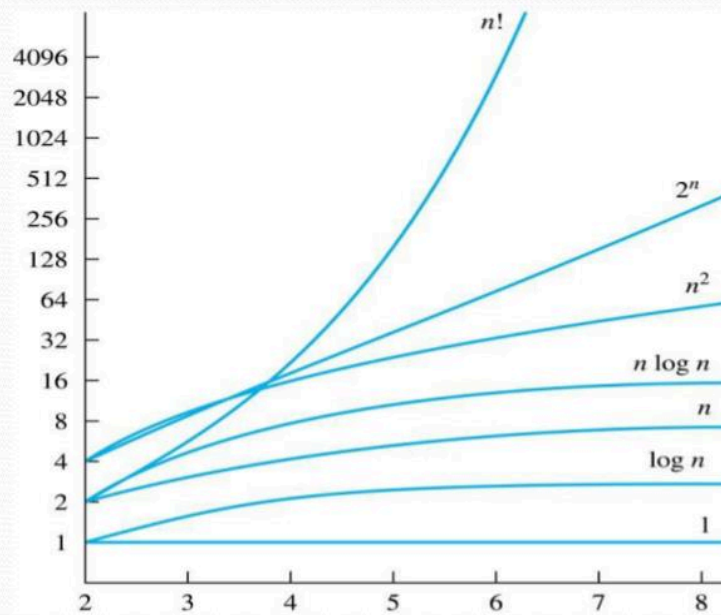
Nested loops → time grows as the square of input size.

Example: Bubble Sort

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1])  
            {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

- $O(\log n)$: Logarithmic (binary search).
- $O(n \log n)$: Linear logarithmic Sorting (merge sort, quicksort).
- $O(2^n)$: Exponential (recursive subset generation).

Display of Growth of Functions



Note the difference in behavior of functions as n gets larger

Order of Growth Comparison

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

1. Best Case Time Complexity

Definition:

The **best-case** time complexity is the minimum amount of time an algorithm takes to complete, given the most favorable input.

This is when the algorithm finishes its task with the least number of operations.

Example – Linear Search

Suppose we have an array:

[2, 4, 6, 8, 10]

We want to search for 2.

In this case, the element is found at the first position, so only one comparison is needed.

Best Case Time Complexity: $O(1)$
(Found immediately)

2. Worst Case Time Complexity

Definition:

The **worst-case** time complexity is the maximum amount of time an algorithm takes to complete, given the most unfavorable input.

This is when the algorithm performs the maximum number of operations before finishing.

Example – Linear Search

Same array: [2, 4, 6, 8, 10]

We search for 10 (which is present in last).

Now, the algorithm must check every element before concluding that 10 is in the last.

Worst Case Time Complexity: $O(n)$

(Needs to check all n elements)

3. Average Case Time Complexity

Definition:

The **average case** time complexity is the expected time an algorithm takes to complete, considering all possible inputs equally likely.

It represents a more realistic scenario of algorithm performance.

Example – Linear Search

If we assume we are equally likely to search for any element:

- Sometimes we find it early (best case)
- Sometimes near the end (worst case)

Average Case Time Complexity: $O(n)$

On average, we'll check about $n/2$ elements.

Asymptotic Notations

- Asymptotic notation is used to describe the running time (or growth rate) of an algorithm as the input size n becomes large.
- It helps us compare algorithms independently of hardware or language.

The three main notations are:

1. Big O (O) → *Upper Bound (Worst Case)*
2. Omega (Ω) → *Lower Bound (Best Case)*
3. Theta (Θ) → *Tight Bound (Average or Exact Case)*

1. Big O Notation – $O(g(n))$

Meaning:

- Represents the upper bound on the running time.
- It gives the worst-case time, the *longest time* taken by the algorithm to complete its execution.
- Ensures the algorithm won't take more time than this bound.

Mathematical Condition:

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

where

- $f(n)$: actual running time
- $g(n)$: comparison function
- c : positive constant
- n_0 : starting point after which the condition holds

 Example:

Let $f(n) = 2n + 3$

We want to prove $f(n) = O(n)$

Check condition:

$$2n + 3 \leq c \cdot n \text{ for } n \geq n_0$$

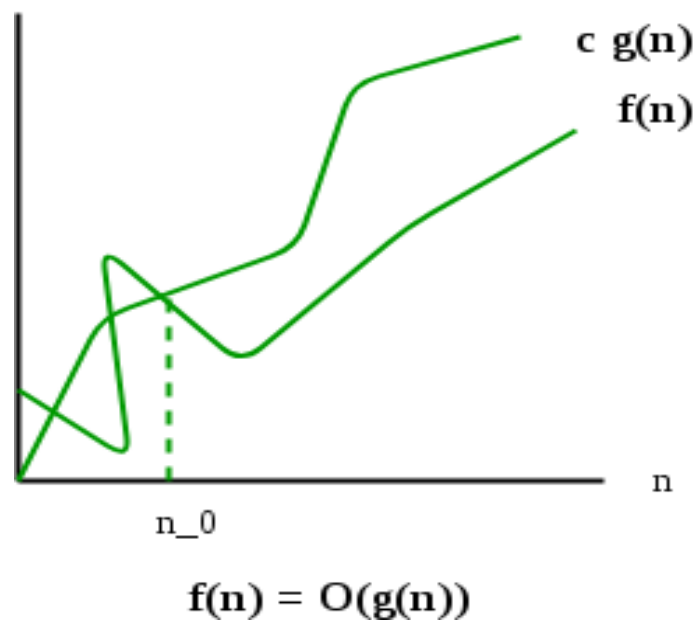
Choose $c = 3, n_0 = 3$

Then, for $n \geq 3$:

$$2n + 3 \leq 3n \Rightarrow \text{Condition holds } \checkmark$$

Hence: $2n + 3 = O(n)$

Graph



2. Omega Notation – $\Omega(g(n))$

Meaning:

- Represents the lower bound on the running time.
- It gives the best-case time — the *shortest time* taken by the algorithm to complete its execution.
- Ensures the algorithm takes at least this much time.

Mathematical Condition:

$$f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0$$

 Example:

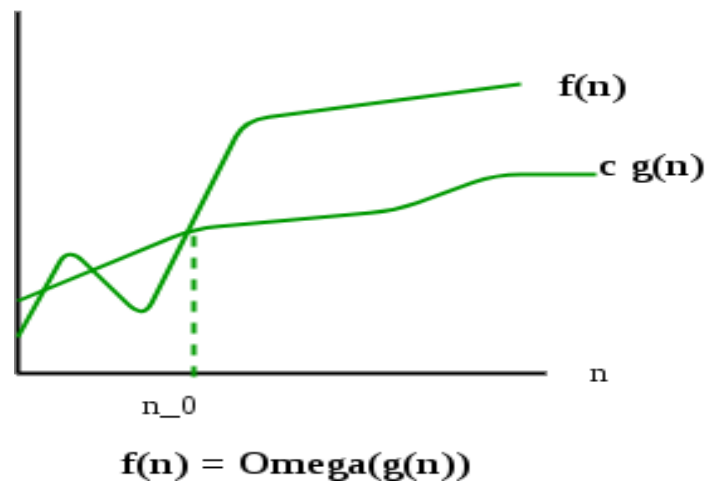
$$f(n) = 2n + 3$$

We can show $f(n) = \Omega(n)$

Choose $c = 2, n_0 = 1$

$$2n + 3 \geq 2n \text{ for all } n \geq 1 \quad \checkmark$$

Hence $f(n) = \Omega(n)$



3. Theta Notation – $\Theta(g(n))$

Meaning:

- Represents the tight bound (both upper and lower bounds).
- It gives the average case or exact growth rate.
- The algorithm's time grows at the same rate as $g(n)$.

Mathematical Condition:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0$$



Example:

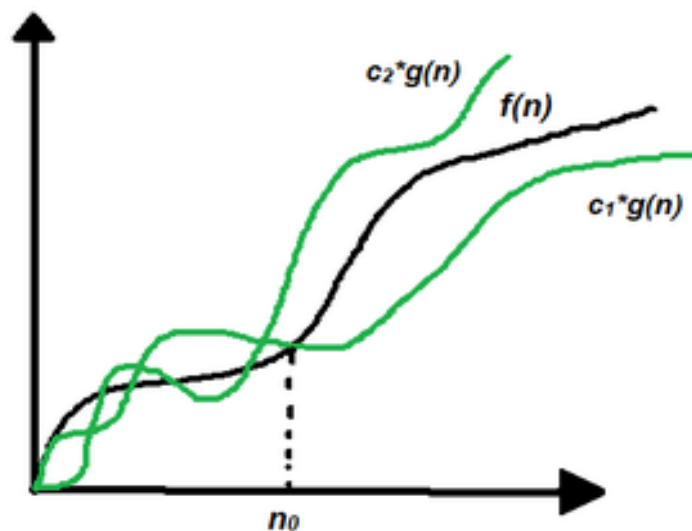
For $f(n) = 2n + 3$, we can show $f(n) = \Theta(n)$

Take $c_1 = 2, c_2 = 3, n_0 = 3$

$$2n \leq 2n + 3 \leq 3n \text{ for all } n \geq 3 \quad \checkmark$$

Hence $f(n) = \Theta(n)$

Graph



What is iteration and recursion

1. Iteration

Definition:

- Iteration means repeating a set of instructions until a certain condition is met.
- It uses loops (like **for**, **while**, or **do-while**) to execute the same block of code multiple times.

Key Idea:

- The loop control variable changes with each repetition.
- The process stops when the condition becomes false.
- Used for repetitive tasks where the number of steps is known or easily determined.

Example (Finding Factorial using Iteration):

```
int factorial(int n) {  
    int fact = 1;  
    for (int i = 1; i <= n; i++) {  
        fact = fact * i;  
    }  
    return fact;  
}
```

2. Recursion

Definition:

- Recursion means a function calling itself directly or indirectly to solve a smaller instance of the same problem.
- It continues until a base condition (or stopping condition) is reached.

Key Idea:

- The problem is divided into smaller subproblems of the same type.
- Each recursive call works on a smaller input.
- The process stops when the base condition is satisfied.

Example (Finding Factorial using Recursion):

```
int factorial(int n) {  
    if (n == 0 || n == 1)  
        return 1;           // Base case  
    else  
        return n * factorial(n - 1); // Recursive call  
}
```

Tower of Hanoi

Definition:

- The Tower of Hanoi is a classical recursive problem in computer science and mathematics.
- It involves moving a set of disks of different sizes from one rod (source) to another rod (destination), using a third rod (auxiliary), following certain rules.

Task / Objective:

You are given:

- Three rods — Source (A), Auxiliary (B), and Destination (C)
- n disks — all disks are of different sizes and stacked on the source rod in decreasing order (largest at bottom, smallest at top).

Goal:

“Move all disks from source rod A to destination rod C, using auxiliary rod B, while following the rules.”

Rules:

1. Only one disk can be moved at a time.
2. You can move only the top disk of any rod.
3. No larger disk can be placed on top of a smaller disk.

Algorithm (Recursive Approach):

T0H(n, A,B,c)

1. If $n == 1$:

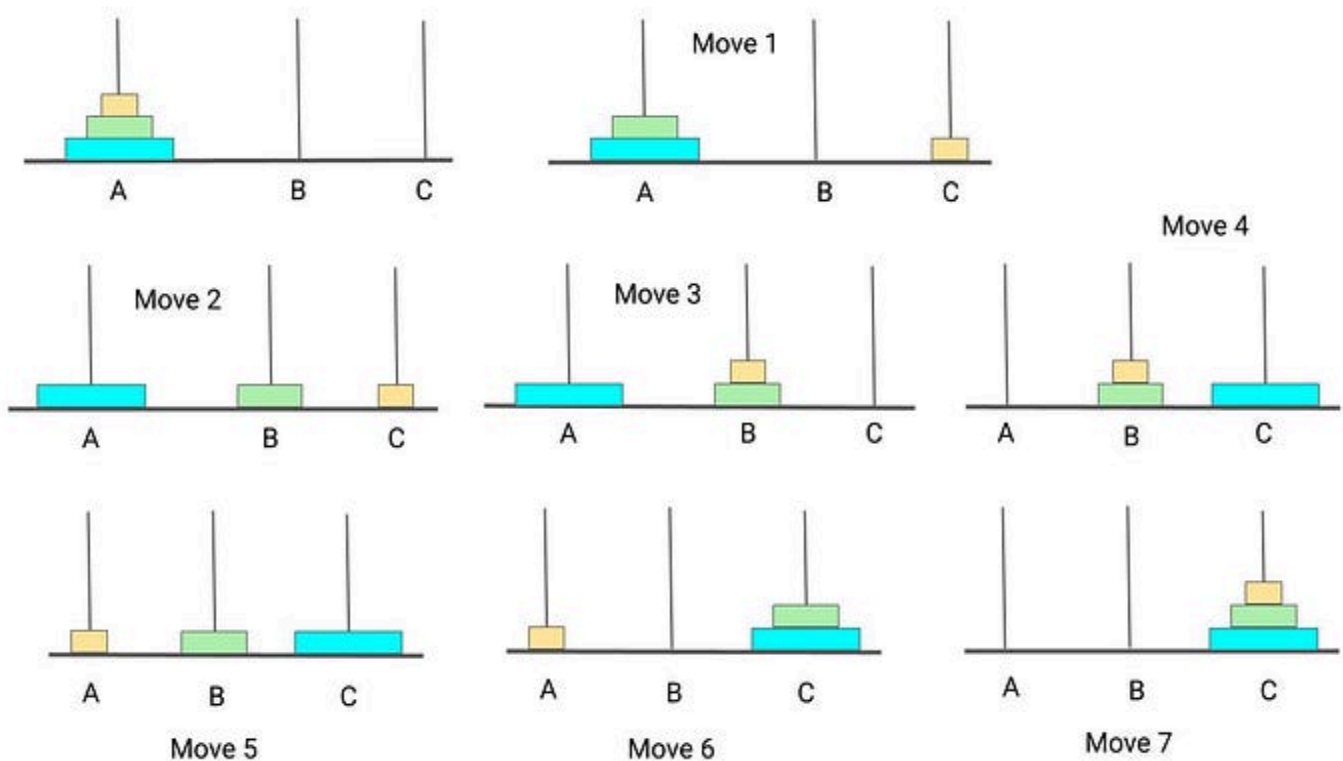
Move disk from A $\rightarrow$ C

2. Else:

T0H(n-1,A,C,B)

Move disk from A $\rightarrow$ C

T0H(n-1,B,A,C)



Minimum number of moves = $2^n - 1$

If $n=3$ then moves is $2^3 - 1 = 7$

